Connor Humphrey OS 4420

# Assignment 06: Majority Voting

## Program Description

For assignment 6, I created a majority voting program using C. The program uses pipes and child processes to simulate a voting system with one judge and multiple voters. The judge is the parent process, while each voter is a child process created using a fork system call. The program first asks the user to enter the number of voters and the maximum number of voting rounds. Each voter randomly chooses either yes or no and sends its vote to the judge through its own pipe. The judge uses the read system call to collect the votes. It counts the total number of yes and no votes, and announces the majority decision for each round. If both sides receive the same number of votes, then the judge announces a tie.

## Process Communication

The program uses one pipe for each voter to communicate with the judge. The parent process creates all the pipes before creating the child processes. Each child process is assigned a voter ID that corresponds to its pipe number. The voters use the write system call to send their votes, while the judge uses the read system call to collect them. All voters participate in every round without creating additional processes. After the maximum number of rounds is reached, the program closes the pipes, and the judge uses waitpid to wait for the child processes to finish. This ensures that all processes terminate properly before the program exits.

## Compiling and Running the Program

I compiled and ran the program using WSL. To compile the program, I used the command gcc -Wall -Wextra -std=c11 majority_voting.c -o majority_voting. After compiling successfully, I ran the program using ./majority_voting. The program asked me to enter the number of voters and the number of rounds. For testing, I entered five voters and three rounds. The program displays each voter's choice and the judge's majority decision for every round. Since the votes are randomly generated, the results may be different each time the program runs.

## Screenshots

```
OS_6 on  main [?] via C v11.4.0-gcc took 2s
❯ gcc -Wall -Wextra -std=c11 majority_voting.c -o majority_voting

OS_6 on  main [?] via C v11.4.0-gcc
```

```
OS_6 on  main [?] via C v11.4.0-gcc
❯ ./majority_voting
Enter the number of voters (at least 3): 5
Enter the maximum number of rounds: 3
Round 1 - Voter 0 (PID 18064): YES
Round 2 - Voter 0 (PID 18064): YES
Round 3 - Voter 0 (PID 18064): YES
Round 1 - Voter 1 (PID 18065): NO
Round 2 - Voter 1 (PID 18065): NO
Round 3 - Voter 1 (PID 18065): YES
Round 1 - Voter 2 (PID 18066): YES
Round 2 - Voter 2 (PID 18066): NO
Round 3 - Voter 2 (PID 18066): YES
Round 1 - Voter 3 (PID 18067): YES
Round 2 - Voter 3 (PID 18067): NO
Round 3 - Voter 3 (PID 18067): NO

===== Round 1 =====
Round 1 - Voter 4 (PID 18068): YES
Round 2 - Voter 4 (PID 18068): YES
Judge: YES votes = 4
Round 3 - Voter 4 (PID 18068): NO
Judge: NO votes = 1
Judge Decision: YES wins!

===== Round 2 =====
Judge: YES votes = 2
Judge: NO votes = 3
Judge Decision: NO wins!

===== Round 3 =====
Judge: YES votes = 3
Judge: NO votes = 2
Judge Decision: YES wins!

All voting rounds completed.
All voter processes terminated.

OS_6 on  main [?] via C v11.4.0-gcc took 2s
```